

**Exercise 8.1: What Are Adversarial Examples? Explain what an adversarial example is. How do they differ from Out-of-Distribution examples?**

- Adversarial examples are inputs intentionally modified with small perturbations that cause the model to make incorrect predictions.
- The perturbations are often imperceptible to humans.
- OOD examples are naturally different inputs that lie outside the training distribution.
- Adversarial examples are maliciously crafted, whereas OOD examples occur naturally.

**Example:**

- Adversarial: Slightly altered sunny image fools the model.
- OOD: Night-time image unseen during training.

**Exercise 8.2: Attack Formulation A basic gradient-based attack updates the input as:**

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha \nabla_{\mathbf{x}} L(y, f(\mathbf{x}_i))$$

**1. What does each term represent?**

| Term                    | Meaning                      |
|-------------------------|------------------------------|
| $\mathbf{x}_i$          | Current input                |
| $\alpha$                | Step size                    |
| $\nabla_{\mathbf{x}} L$ | Gradient of loss w.r.t input |
| $L(y, f(\mathbf{x}_i))$ | Model loss                   |
| $\mathbf{x}_{i+1}$      | Updated adversarial input    |

**2. What is the difference between a targeted and an untargeted attack? How would the formula change for a targeted attack?**

- Untargeted attacks aim to make the prediction incorrect by maximizing loss.  
*Correct  $\rightarrow$  Wrong*
- Targeted attacks aim to force a specific prediction by minimizing the target loss.  
*Correct  $\rightarrow$  Specific Wrong Class*
- Targeted attacks reverse the update direction.

Change into:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \alpha \nabla_{\mathbf{x}} L$$

**3. Why does this basic update rule not respect a perturbation budget  $\|\mathbf{x}_0 - \mathbf{x}_t\| \leq \epsilon$ ? How can it be modified to do so?**

- The basic update rule has no perturbation limit.
- The adversarial example may move far from the original image.
- PGD projects the perturbed input back into the  $\epsilon$ -budget after each update.

$\epsilon$  makes sure the cheating stays invisible.

### Exercise 8.3: Defenses

Explain the idea of adversarial training. What trade-off does it introduce?

- Adversarial training improves robustness by training the model using adversarial examples alongside clean examples.
- The model learns to perform well even under worst-case perturbations.
- Trade-offs include increased computational cost and reduced accuracy on clean data.
- Robustness often generalizes only to attacks similar to those used during training.

Example:

Training a pedestrian detector using FGSM examples makes it harder to fool, but may slightly reduce its normal performance.

### Exercise 8.4: Generating Adversarial Examples

1. Implement FGSM for each of your three binary classifiers.

2. Generate adversarial examples for  $\epsilon \in \{0.01, 0.05, 0.1\}$ .

3. Display a clean image and its adversarial counterpart side-by-side. At what  $\epsilon$  do the perturbations become visible to a human?



- At  $\epsilon = 0.01$ , the perturbations were not visible to humans.
- At  $\epsilon = 0.05$ , slight noise became noticeable while the scene remained understandable.
- At  $\epsilon = 0.1$ , the perturbations were clearly visible and significantly altered the appearance of the image.

### Exercise 8.5: Measuring Robustness

1. Evaluate each of your three models on the adversarial test set for each  $\epsilon$ . (You may also use a subset of 100 randomly sampled images for evaluation.) Report the per-model recall drop compared to clean inputs.

**PED:**

| $\epsilon$ | Recall | Recall Drop |
|------------|--------|-------------|
| Clean      | 0.8363 | -           |
| 0.01       | 0.3540 | 0.4823      |
| 0.05       | 1.0000 | -0.1637     |
| 0.10       | 1.0000 | -0.1637     |

- Recall dropped to 0 at  $\epsilon = 0.01$ , indicating that small perturbations caused the model to miss all pedestrians.
- At larger  $\epsilon$  values, recall increased again.
- The increase suggests that the model became biased toward predicting the positive class more frequently, resulting in unstable behaviour under attack.

**VEHICLE:**

| $\epsilon$ | Recall | Recall Drop |
|------------|--------|-------------|
| Clean      | 0.8363 | -           |
| 0.01       | 0.3540 | 0.4823      |
| 0.05       | 1.0000 | -0.1637     |
| 0.10       | 1.0000 | -0.1637     |

- Recall decreased substantially at  $\epsilon = 0.01$ .
- For  $\epsilon = 0.05$  and  $\epsilon = 0.10$ , the model predicted "vehicle" for all inputs, resulting in a recall of 1.0.
- Although recall appeared perfect, this behaviour indicates excessive false positives and poor robustness.

**TRAFFIC:**

| $\epsilon$ | Recall | Recall Drop |
|------------|--------|-------------|
| Clean      | 0.9776 | -           |
| 0.01       | 0.3363 | 0.6413      |
| 0.05       | 0.0135 | 0.9641      |
| 0.10       | 0.0538 | 0.9238      |

- The traffic-light model showed the largest degradation in recall under FGSM attacks.
- Even a small perturbation ( $\epsilon = 0.01$ ) significantly reduced recall.
- The model was highly vulnerable to adversarial perturbations.

**Exercise 8.6: Extending the Safety Analysis for Adversarial Robustness** You have now measured how much adversarial perturbations degrade your models' recall. Revisit your System-Theoretic Process Analysis from Sheet 2 and extend it to cover this threat. Many of you will not yet have an adversarial-attack entry in your analysis; the goal of this exercise is to add one and trace it through.

**1. Hazard.** Check the hazards from Exercise 2.4. Is a degraded perception output caused by an adversarial input captured (e.g. as a cause of “vehicle fails to brake for a pedestrian”)? If your hazard list does not reflect it, refine or add a hazard.

Our original hazards covered failures such as the vehicle failing to brake for a pedestrian. However, they did not explicitly consider that these failures could be caused by adversarial inputs.

| ID  | Hazard                                                                                                                             | Loss(es)      | Likelihood | Severity |
|-----|------------------------------------------------------------------------------------------------------------------------------------|---------------|------------|----------|
| H-6 | The system relies on incorrect perception outputs caused by adversarially perturbed inputs, resulting in unsafe vehicle behaviour. | L-1, L-2, L-3 | Medium     | High     |

**Example:** An adversarial perturbation causes the pedestrian detector to miss a pedestrian, leading to the vehicle not braking.

**2. Unsafe control action.** Review the UCAs from Exercise 2.6. Add a UCA for the case where the planner acts on perception output that is wrong because the input has been adversarially perturbed and this is not detected (e.g. “the planner continues at speed while the pedestrian classifier has been fooled by an adversarial attack and outputs a false negative”). Link it to the hazard(s) above.

| ID     | Controller      | Control Action   | UCA Type          | Hazard(s) | Unsafe Scenario                                                                                                                                |
|--------|-----------------|------------------|-------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| UCA-10 | Planning Module | Continue Driving | Provided Unsafely | H-6       | The planner continues normal driving while the perception output has been corrupted by an adversarial attack and this failure is not detected. |

Example: The pedestrian classifier outputs a false negative due to an adversarial attack, but the planner trusts the prediction and continues driving.

**3. Safety constraints. From that UCA, derive at least one new safety constraint of each kind (cf. Exercise 2.7): • a model-level constraint on adversarial robustness, referencing the  $\epsilon$  budget and recall drop you measured; and • a system-level constraint on the response when a low-confidence or anomalous prediction is detected.**

| UCA    | Safety Constraint                                                                                                                                                                      | Level       |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| UCA-10 | The perception models shall maintain acceptable robustness against adversarial perturbations up to a predefined $\epsilon$ budget and shall limit the recall degradation under attack. | Model-Level |

**Justification using our results:**

- The traffic-light model showed a severe recall drop from 0.978 (clean) to 0.013 ( $\epsilon = 0.05$ ).
- Therefore, adversarial robustness requirements should be defined and tested for expected perturbation budgets.

**4. Residual risk. Even with adversarial training that meets your model-level constraint, what risk remains? Does robust training fully address the UCA and hazard?**

**Even with adversarial training, some risk remains.**

- Adversarial training only improves robustness for perturbations similar to those used during training.
- New attack strategies or stronger perturbations may still fool the model.
- The vehicle model showed that metrics such as recall can be misleading, as recall reached 1.0 because the model predicted the positive class for all inputs.
- Detection mechanisms and safe fallback behaviours are still required.
- Therefore, adversarial training reduces risk but does not completely eliminate the hazard.